

บทที่ 5

การทดลองและผลการทดลอง

ในบทนี้จะกล่าวถึงส่วนของการทดลองนั้นในโครงการนี้จำเป็นอย่างยิ่งในการทดลอง โดยการทดลองนั้นได้ทำการทดลองตามแต่ละวิธีของการเจาะระบบผ่านเว็บซึ่งแต่ละวิธีมีความแตกต่างกันในการทำการทดลองซึ่งมีค่าไปในการทดลองให้ด้วย

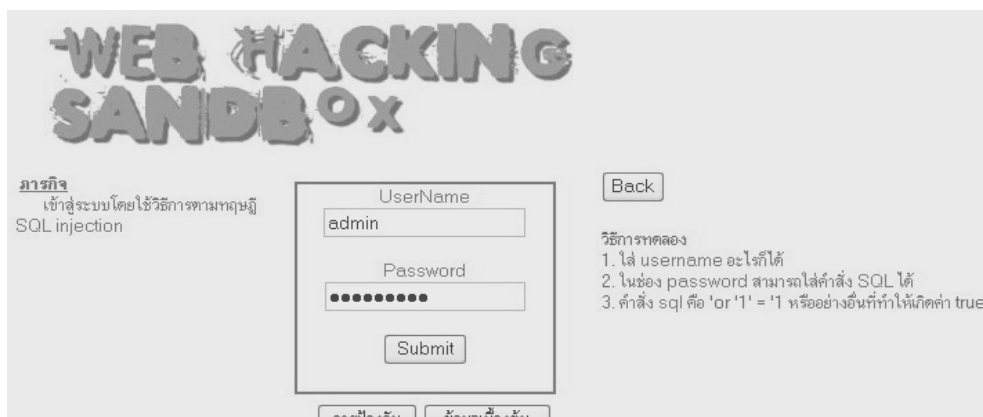
5.1 SQL Injection

การทดลอง

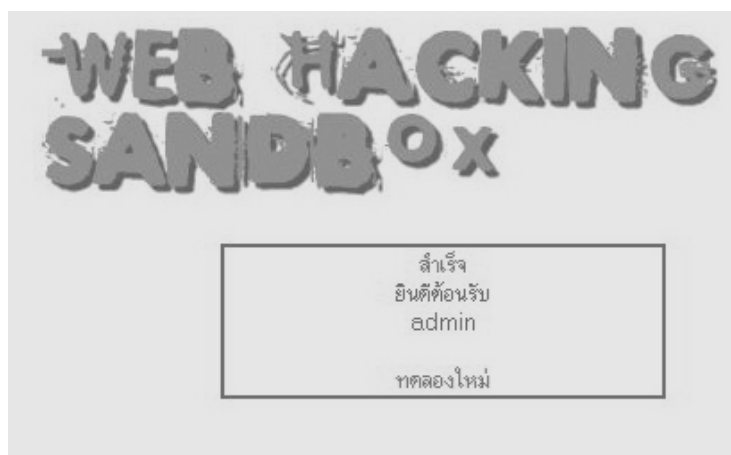
สามารถทำได้ทุกหน้าที่มีการติดต่อกับดาตาเบสแต่ที่นำมาทดลองนั้นเป็นการผ่านหน้าการพิสูจน์ตัวตนเข้าสู่เว็บได้โดยการใช้รูปแบบ SQL Statement เช่น ' or '1' = '1' หรือ " or "1"= "1" ตัวอย่างการทำ SQL Injection มีดังรูปต่อที่ 5.1, รูปที่ 5.2 และ รูปที่ 5.3



รูปที่ 5.1 หน้า Login ในเทคนิค SQL Injection



รูปที่ 5.2 การผ่านระบบ login ด้วย SQL Statement



รูปที่ 5.3 การ login ระบบผ่านด้วย SQL Injection สำเร็จ

การป้องกัน



รูปที่ 5.4 การป้องกันการทำ SQL Injection

จากรูปที่ 5.4 ได้ทำการกรอง Statement SQL ไว้แล้วจึงไม่สามารถทำ SQL Injection ได้ซึ่งทำได้ด้วยการกรองตัวอักขระพิเศษต่างๆ ซึ่งถ้าเป็นใน PHP จะมีฟังก์ชันแบบ build-in มาให้แล้วคือ Function addslashes

5.2 Hidden Manipulation

การทดลอง

สามารถทำได้โดยการเปลี่ยนแปลงค่าใน hidden field ให้เป็นค่าที่ต้องการแล้วก็ทำการส่งไปประมวลที่ Server โดยการทดลองนั้นได้ดังรูปที่ 5.5 และ รูปที่ 5.6



รูปที่ 5.5 การสั่งซื้อสินค้าจำนวน 100 ชิ้นซึ่งไม่ได้ทำการแก้ราคา



รูปที่ 5.6 การสั่งซื้อสินค้าจำนวน 100 ชิ้นในราคา 1 บาท

ขั้นแรกทำการ Save page ที่ต้องการจะทำแล้วเปิดด้วยโปรแกรม notepad หรือโปรแกรม editor ทั่วไปได้ ซึ่งตัวอย่างของโค้ดของ form ที่ 4.1 มีดังนี้

(5.1)

```
<form name="form2" method="post" action="javack.php">
<input name="price" type="hidden" id="price" value="5000">
<p><span class="style2">ซื้อจำนวน</span>
<input name="textfield" type="text" size="16" maxlength="16" >
<span class="style2">ตัว</span></p>
<p>
```

```
<input type="submit" name="Submit" value="Submit">
</p>
</form>
```

ซึ่งการทำ Hidden Manipulation นั้นทำได้โดยการแก้ไข input ที่มี type เป็น hidden field โดยการแก้ไขค่า value ="x" ให้น้อยลงจากราคาปกติตามที่ Hacker ต้องการ ซึ่งเมื่อทำการแก้ค่าแล้ว สิ่งต่อไปก็เป็นการส่งค่าที่ทำการเปลี่ยนนั้นไปประมวลผลทาง Server การเปลี่ยน ในส่วนของ form ที่ทำการส่ง form นั้นไปยังฝั่ง Server ซึ่งส่วนที่ทำหน้าที่นี้คือ action= "www.example.com/javack.php" เท่านี้ก็เป็น การส่งหน้า page นี้ไปยัง Server ได้แล้วซึ่งถ้า website นั้นไม่มีการป้องกันการ ทำ Hidden Manipulation เอาไว้สิ่งที่ได้ทำไปก็จะ ได้ผล

การป้องกัน



รูปที่ 5.7 การสั่งซื้อสินค้าจำนวน 100 ชิ้น

ใช้การ Check ค่า ตัวแปร HTTP_REFERER ซึ่งเป็นตัวแปร Environment Variable ของ HTTP เพื่อดูว่าหน้า page ที่ส่งค่ามานั้นมาจากหน้าเว็บของ Server ของ เว็บของเราหรือไม่ วิธีนี้ก็ยังไม่น่าปลอดภัยเพราะว่า Hacker สามารถที่จะเขียนส่วนหัวของ Header ของ HTTP ให้สามารถ

เปลี่ยนค่า HTTP_REFERER ได้เช่นกัน การที่จะให้ปลอดภัยมากขึ้นก็ต้องมีการ Check ค่าที่รับมาว่าค่านั้นถูกต้องก่อนการคำนวณราคาสินค้าและต้องมีการ Check ขนาดของค่าที่รับมาว่ามีขนาดไม่เกินที่กำหนดไว้หรือไม่ด้วย ลดหรือไม่ใช้ input type hidden ในการรับส่งค่าสำคัญๆ ต่างๆซึ่งในPHPนั้นสามารถใช้ตัวแปร SESSION ซึ่งตัวแปร SESSION นั้นจะเก็บเอาไว้ทางฝั่งของ Server ซึ่งจะปลอดภัยกว่าแต่ก็ใช่ว่าตัวแปร SESSION จะปลอดภัยเสียทีเดียวแต่ก็ปลอดภัยกว่า การส่งตัวแปรแบบ Hidden ดังรูปที่ 5.7 แสดงให้เห็นว่ามีการป้องกันแล้ว

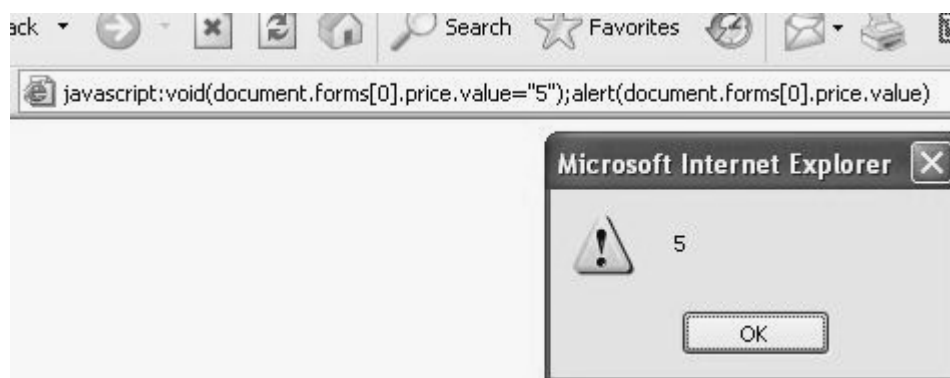
5.3 Java Injection

การทดลอง

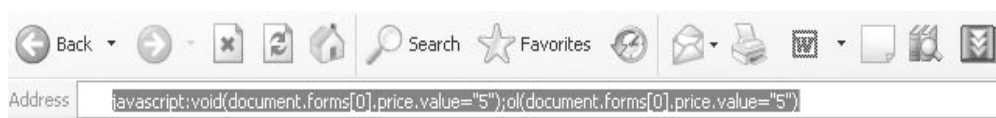
Java Injection มีสองประเภทคือ Java Injection Form Editing และ Java Injection Cookie Editing

5.3.1 Java Injection Form Editing

ทำได้โดยการview ดูโค้ดเพื่อดูลำดับของ form และ field ที่ต้องการแก้ไขแล้วใช้คำสั่ง Javascript ที่ URL ดังรูป



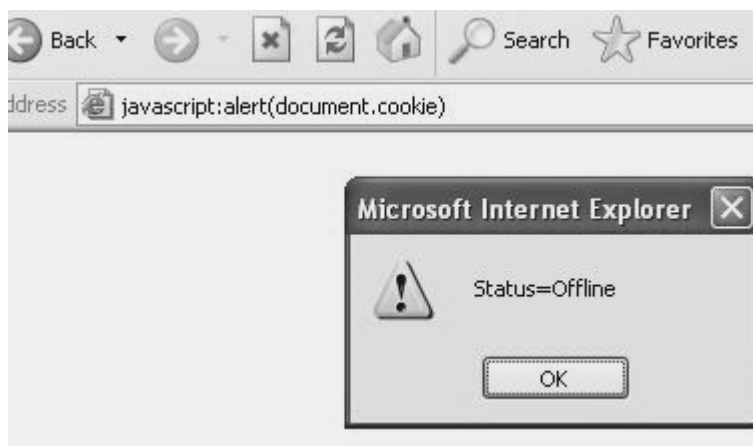
รูปที่ 5.8 การเปลี่ยนค่า field ภายในเครื่อง Client และดูค่าที่เปลี่ยน



รูปที่ 5.9 การส่งค่าที่เปลี่ยนไปประมวลผลที่ Server

5.3.2 Java Injection Cookie Editing

สามารถเปลี่ยนค่าของ Cookie โดยใช้ Javascript ในการเปลี่ยนค่า Cookie โดยทำได้ดังรูป



รูปที่ 5.10 การดูค่า Cookie

จากรูปที่ 4.8 เป็นการแสดง cookie นั้นเองส่วนการทำ Injection นั้นก็ไม่ยากโดยการใช้คำสั่งเดิมแต่ทำการเปลี่ยนค่า cookie ตามที่เราต้องการรูปแบบการทำได้คือ

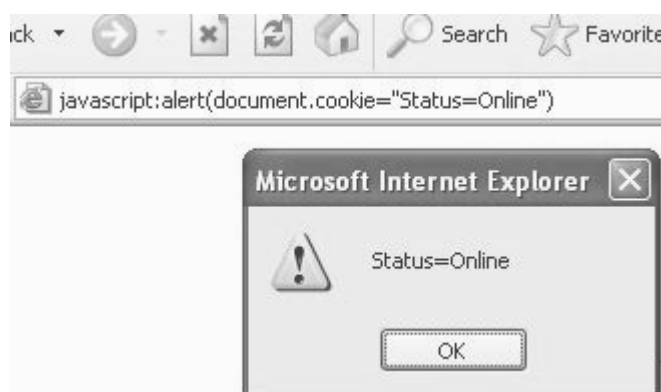
(5.2)

```
javascript:void(document.cookie="Field = myValue");
```

Field คือตัวแปรของ cookie ส่วน myValue นั่นคือค่า cookie นั้นเองมาลองดูตัวอย่างการทดลองดังรูปที่ 5.11

(5.3)

```
javascript:void(document.cookie="Status = Online");
```



รูปที่ 5.11 เป็นการเปลี่ยนค่า Cookie

เป็นการเปลี่ยน cookie ที่มีชื่อ Status เป็น Online ซึ่งถ้าเว็บบางเว็บนั้นเก็บข้อมูลสำคัญลงใน cookie เช่นสถานการณ์ login เป็นต้นเราก็สามารถใช้วิธี

การป้องกัน



รูปที่ 5.12 ค่า Cookie ซึ่งไม่การเก็บค่า Cookie ที่เป็น Clear text



รูปที่ 5.13 การทำ Java Injection แต่ถูกป้องกันเอาไว้แล้วจึงไม่ alert ขึ้นมา



รูปที่ 5.14 การสั่งซื้อสินค้าที่ไม่มีการใช้ Hidden field ในการเก็บราคา ราคาจึงไม่เปลี่ยน

จากรูปได้มีการหลีกเลี่ยงการเก็บ cookie ที่สำคัญและ การใช้ form ก็ควรระมัดระวังโดยการกรอง input โดยระเอียดและไม่ใช้ Hidden field ในการส่งข้อมูลสำคัญ ถ้าจะใช้ cookie ก็เก็บข้อมูลส่วนที่ไม่สำคัญแล้วก็ต้องเข้ารหัส cookie ด้วย

5.4 Cross Site Script (XSS)

การทดลอง

มีการจำลองเว็บบอร์ดที่สามารถวาง Script ได้ซึ่งผู้ใช้งานสามารถที่จะวาง Script ต่างได้ และจะมีหน้า page ให้ป้องกันด้วยซึ่งไม่สามารถวาง Script ได้มีการป้องกันไว้แล้ว

รูปที่ 5.15 การทดสอบอย่างง่ายว่าเว็บบอร์ดวาง Script ได้หรือไม่

Cross-Site Scripting Test					
คลิกที่นี่เพื่อตั้งกระทู้ใหม่					
แสดงหน้าที่ : <u>1</u>					
No.	หัวข้อ	ผู้เขียน	วันที่เขียน	ผู้ออกความเห็น	ip
196 :	test script	test	3 ม.ค. 2549 - 09:31:01	ผู้ออกความเห็น 0 คน	127.0.0.1

รูปที่ 5.16 การวางกระทู้เพื่อทดสอบ Script



รูปที่ 5.17 กระทู้ที่ได้วางไว้

แสดงให้เห็นว่าเว็บบอร์ดนี้สามารถวางกระทู้ได้เพราะคำว่า test กลายเป็นตัวหนาไปแล้ว และไม่แสดง tag HTML

Webboard

สามารถใส่ Script ต่างๆ ลงไปในบอร์ดได้

หัวข้อ * ::

test script

รายละเอียด * ::

<script>alert(document.cookie)</script>

ชื่อ * ::

test

E-mail ::

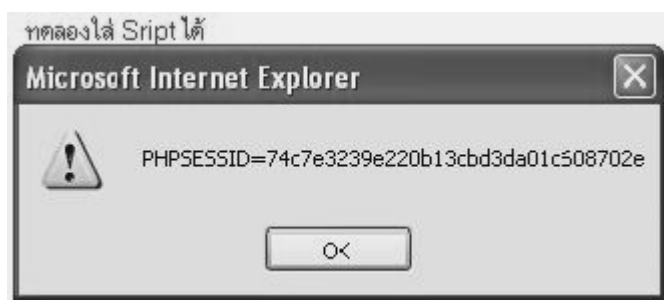
MSN ::

ตั้งกระทู้

ยกเลิก

สามารถใส่ Script ต่างๆ ลงไปในบอร์ดได้

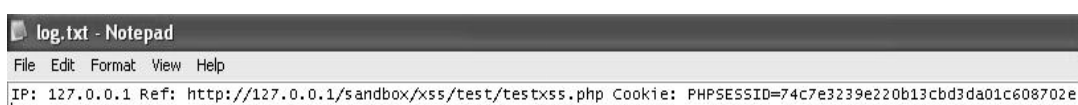
รูปที่ 5.18 การวางกระทู้ที่จะแสดงค่า Cookie



รูปที่ 5.19 ค่า Cookie เมื่อคลิกกระทู้



รูปที่ 5.20 กระทู้ที่ได้วาง Script เพื่อขโมย Cookie



รูปที่ 5.21 ค่า Cookie ที่ถูกขโมย

การป้องกัน



รูปที่ 5.22 กระทู้ที่ได้วางไว้แต่เว็บบอร์ดไม่สามารถวาง Script ได้

การป้องกันก็มีการตรวจเช็คอินพุตที่เป็นอักขระพิเศษต่างๆก่อนรับเข้ามาซึ่งในโครงการได้ใช้ PHP เขียน โดย PHP ได้มีฟังก์ชันที่สามารถตรวจเช็คพวกอักขระพิเศษคือ ฟังก์ชัน htmlspecialchars() ซึ่งในโค้ดจะเป็นดังนี้

(5.4)

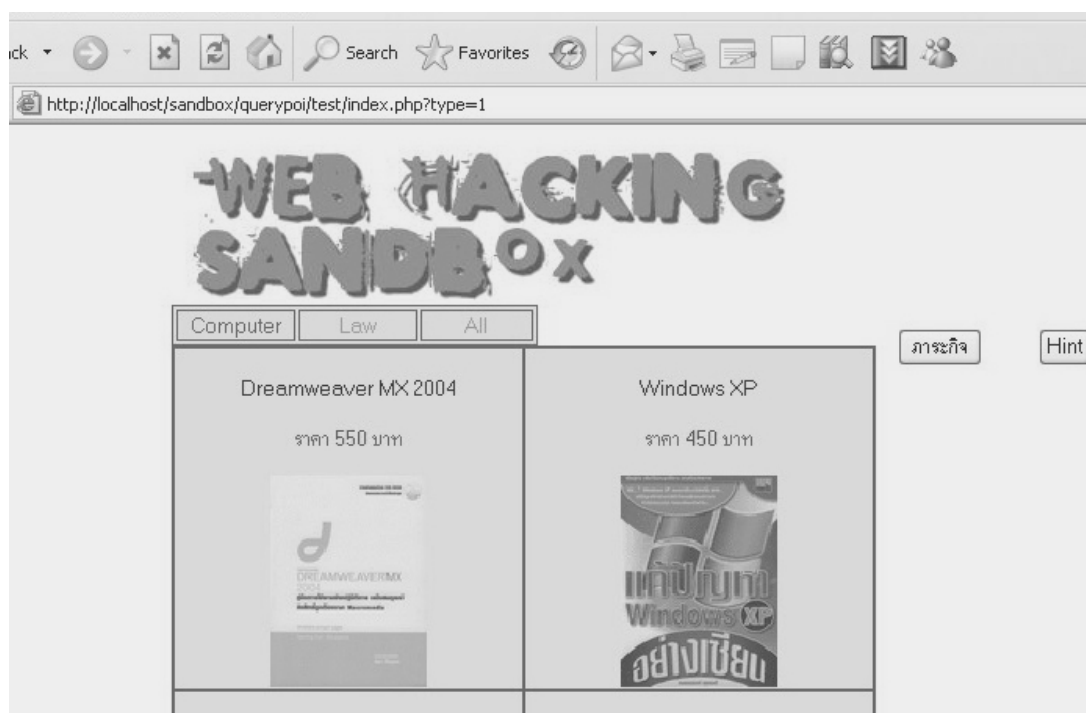
```
$a_message = htmlspecialchars($a_message);
$a_name = htmlspecialchars($a_name);
$a_email = htmlspecialchars($a_email);
$a_icq = htmlspecialchars($a_icq);
```

เป็นการกรองตัวอักขระพิเศษใน message , name, email, icq ก็จะทำให้ไม่สามารถรันสคริปต์ได้ กลายเป็น text file ธรรมดาเท่านั้น

5.5 Query Poisoning

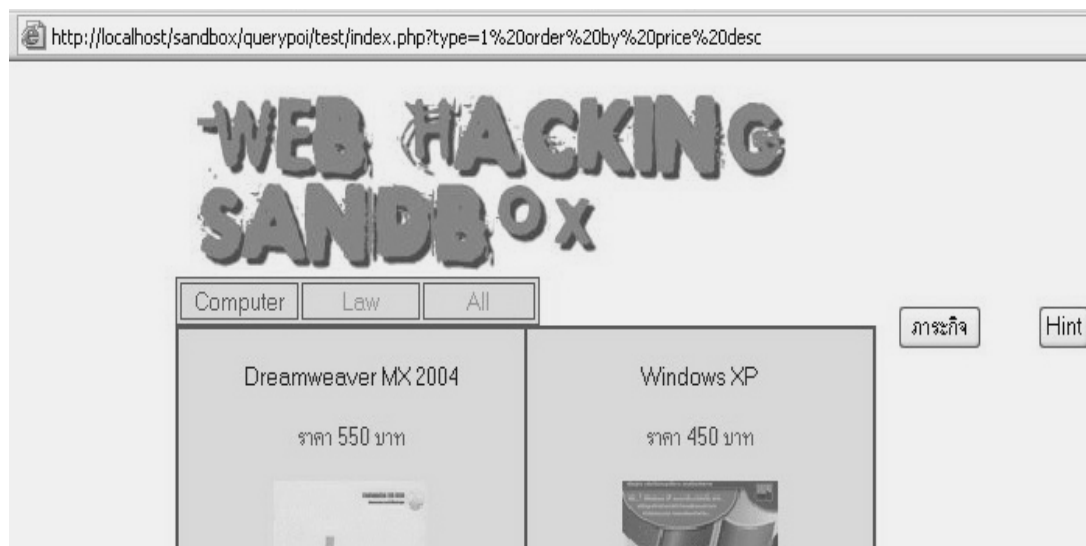
การทดลอง

เป็นการจำลองร้านขายหนังสือผ่านเว็บ ซึ่งสามารถทำ Query Poisoning ได้ ส่วนการป้องกันนั้นมีการกรองการทำ Query Poisoning



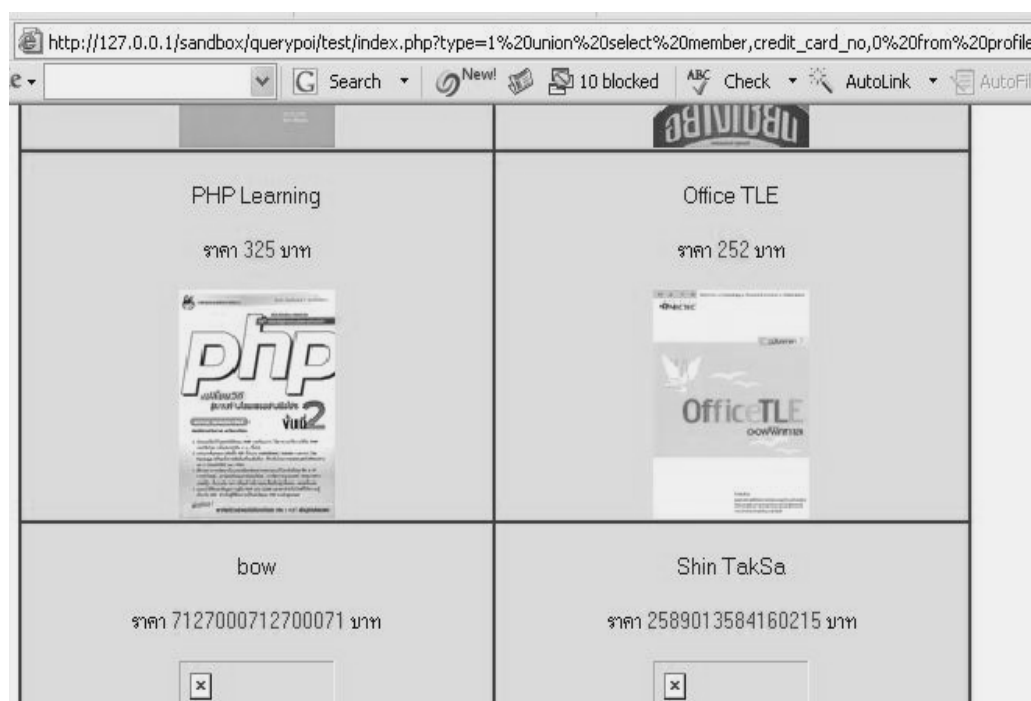
รูปที่ 5.23 การทดสอบการทำ Query Poisoning

เป็นการทดสอบการทำ Query Poisoning โดยการใช้ Statement SQL ฉีดยาเข้าไปใน Query String โดยใช้ Statement “OR 1”



รูปที่ 5.24 ทำ Query Poisoning โดยใช้ Statement “ORDER BY Price DESC”

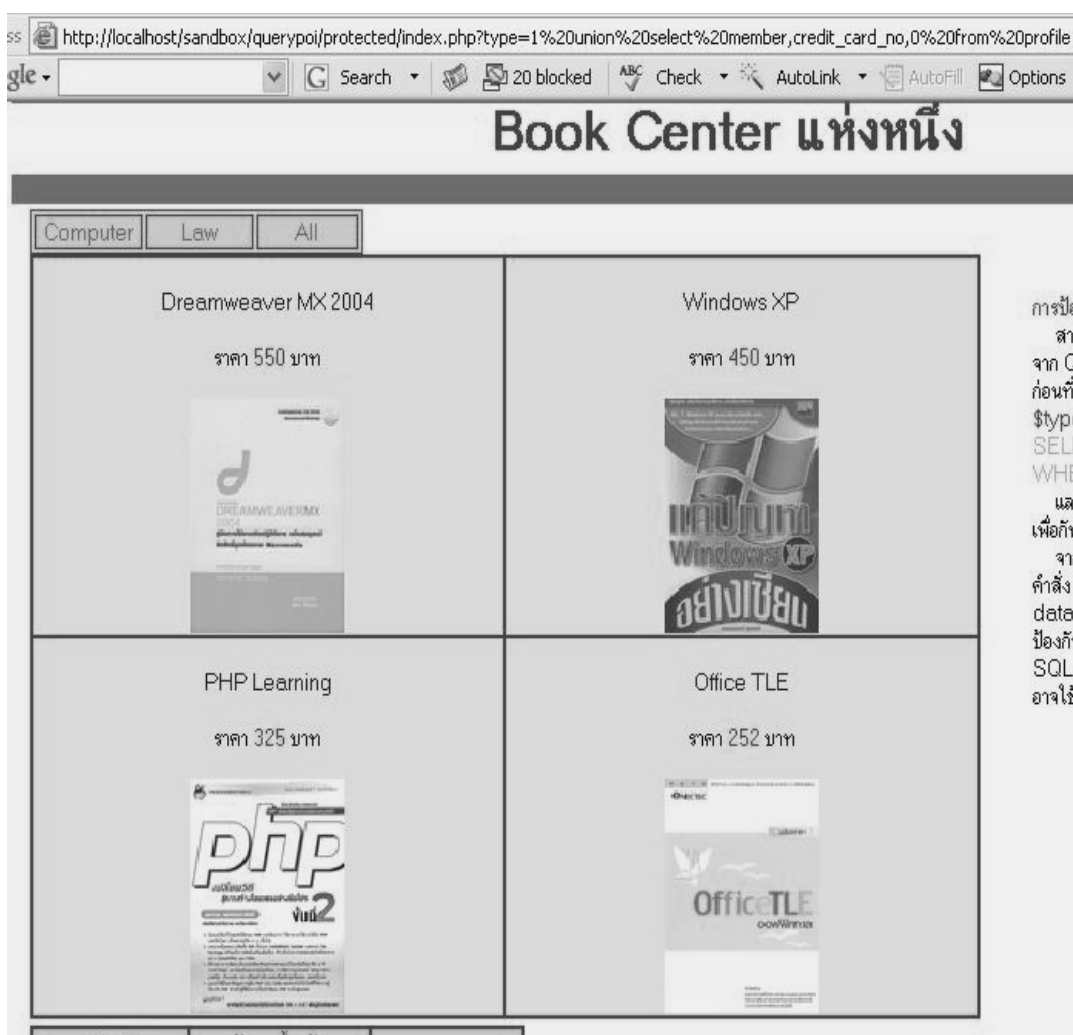
เป็นการเรียงลำดับสินค้าจากราคามากไปหาน้อยแต่ต้องรู้ว่า field ราคาเสียก่อนถึงจะทำได้



รูปที่ 5.25 ทำ Query Poisoning โดยใช้เทคนิคการ UNION

เป็นการใช้ Statement “UNION SELECT member,credit_card_no,0 from profile” เพื่อจะแสดงรายชื่อ สมาชิกเว็บนี้และแสดงหมายเลข Credit Card ของสมาชิกด้วยส่วอีก field ที่ไม่รู้ก็ให้ใส่ 0 หรือ NULL ก็ได้

การป้องกัน



รูปที่ 5.26 การป้องกันการทำ Query Poisoning

จากรูปจะไม่สามารถทำ Query ได้เพราะได้ทำการกรอง Input ที่รับมาให้ดีเช่น พวกตัวอักขระพิเศษต่างๆซึ่งใน PHP นั้นจะ Build in Function addslashes มาให้แล้วฉะนั้นพวกอักขระพิเศษก็ไม่มีปัญหาตามด้วยการใส่ Function intval() เพื่อกรองค่าที่รับมาจาก Query string ก็จะช่วยให้ดีขึ้นหนึ่งแต่ที่น่าเป็นห่วงคือว่า Hacker มักจะเข้ามาทาง Unicode ของตัวอักขระพิเศษแทนก็ต้องเขียนตรวจสอบด้วยเช่น พวก ; ซึ่งใน PHP มีการป้องกันแต่ถ้าเป็น Unicode ยังไม่ได้ป้องกันซึ่งถ้าเป็น Unicode จะเป็น %3B ก็จะเข้ามาได้

5.6 Session Hijacking

การทดลอง

เป็นการจำลองเว็บดาวโหลดแห่งหนึ่งซึ่งผู้ใช้งานจะได้แค่ User ที่เป็นระดับธรรมดาเท่านั้น ซึ่งจะไม่สามารถดาวโหลดไฟล์ได้ซึ่งผู้ใช้งานจะต้องทำการขโมย Session ของสมาชิก VIP ที่สามารถดาวโหลดได้ทุกไฟล์ซึ่งมีรายชื่อสมาชิกและอีเมลซึ่งทางเว็บได้เปิดเผยเอาไว้ ตัวอย่างของการทดลองนี้เป็น Generate Session ที่ไม่ดีที่สามารถคาดเดาได้ ซึ่งในการทดลองนั้นเป็นการจำลองเว็บดาวโหลดแห่งหนึ่งที่ทำให้ดาวโหลดไฟล์ต่าง โดยกำหนดให้ผู้ใช้งานมี User เป็นธรรมดา ซึ่งไม่สามารถดาวโหลดได้ ผู้ใช้งานจะต้องขโมย Session ของ User VIP ที่สามารถดาวโหลดได้ ซึ่ง Session ที่ต้องการให้ขโมยเป็น User ที่ชื่อ trudy โดยในเว็บนี้มีการ Generate Session ขึ้นมาแบบง่ายๆ ซึ่งสามารถคาดเดาได้หากสมัครสมาชิกหลายๆครั้งก็จะคาดเดาได้แล้วดู Session id ก็จะสามารถคาดเดาได้ โดยทางเว็บได้ Generate Session โดยใช้หลักการเข้ารหัสง่ายโดยใช้หลักการ XOR คือถ้ามี Plaintext ที่ต้องการเข้ารหัสก็ต้องมี key ที่ใช้ในการเข้ารหัสเมื่อทำการ XOR กันระหว่าง Plaintext กับ key ก็จะได้ Cipher text ก็คือข้อความที่เข้ารหัสนั่นเอง เมื่อทำการเข้ารหัสแล้วเราก็จะต้องทำการถอดรหัสที่ได้เข้าไว้เพื่อจะเอาไปใช้งาน ซึ่งการทําก็จะทำแบบเดิมเพียงแต่เอา Cipher text มา XOR กับ key เดิมก็จะได้ Plaintext ออกมา ซึ่งในเว็บได้มีการ Generate Session โดยการเอาอีเมลของสมาชิกมาทำการเข้ารหัสโดยคีย์ที่ใช้ จะเป็น AA เป็น key ในการเข้ารหัส Session ซึ่งจะเอาอีเมลมาเปลี่ยนเป็นรหัส ASCII ก่อนในรูปของเลขฐาน 10 ก่อนแล้วเปลี่ยนมาเป็นเลขฐาน 16 แล้วมาทำการ XOR กับ key เช่น รหัส ASCII เป็นเลขฐาน 16 คือ 74 แล้วก็มาทำการ XOR กับ CB คือ 7 XOR A และ 4 XOR A จะได้ de ซึ่งถ้าทำการ XOR ไปทีละ byte จนหมดก็จะได้ Session id มาซึ่งถ้าเป็นของ trudy มีอีเมลเป็น trudy@hotmail.com จะได้ Session เป็น ded8dfced3eac2c5dec7cbc3c684c9c5c7



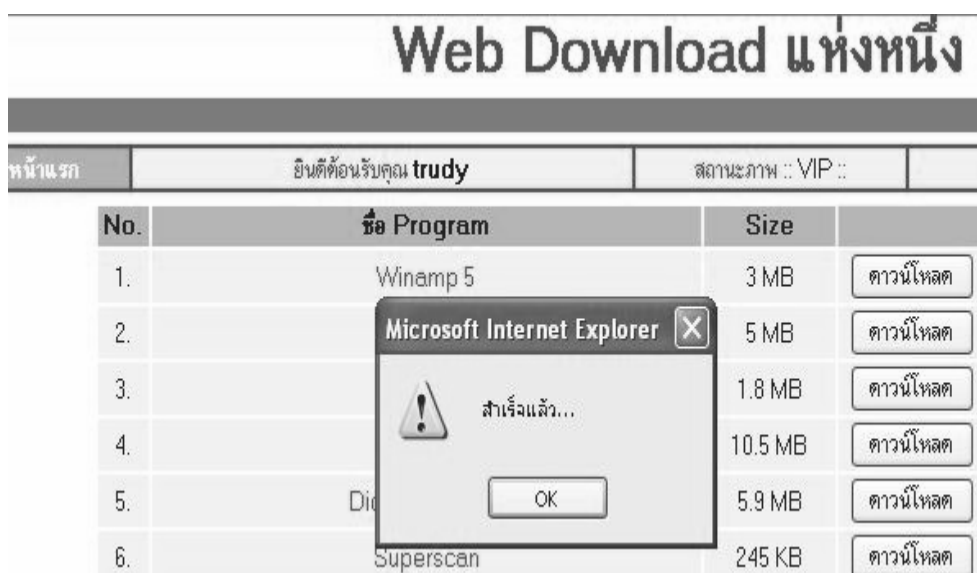
รูปที่ 5.27 หน้าเว็บดาวโหลดที่ Login ด้วย User ธรรมดา



รูปที่ 5.28 ผลของ login ด้วย User ธรรมดาซึ่งไม่สามารถดาวน์โหลดได้



รูปที่ 5.29 หน้าการเปลี่ยนแปลงค่า Session ไปเป็น User ชื่อ Trudy



รูปที่ 5.30 หน้าที่ได้ทำการขโมย Session ของ Trudy ได้เป็นผลสำเร็จ

การป้องกัน

Session Hijacking
การป้องกัน

1. ตัวเลขระบุเซสชัน(Session Identifier)ควรเป็นค่าเฉพาะตัวที่ไม่ซ้ำกัน
เนื่องจากในทุกๆเว็บแอปพลิเคชัน ลอจิคอลเซสชันจำเป็นต้องถูกสร้างขึ้นมาระหว่างเบราว์เซอร์และเว็บเซิร์ฟเวอร์ ดังนั้น เราจึงจำเป็นต้องมีตัวเลขเฉพาะตัวหนึ่งที่เราเรียกว่า ตัวเลขระบุเซสชัน (Session Identifier) เพื่อใช้อ้างถึงเซสชันนั้นๆและตัวเลขดังกล่าว ต้องถูกส่งไปกลับระหว่างเบราว์เซอร์และเว็บเซิร์ฟเวอร์ด้วย หากเป็นไปได้ ทุกๆเซสชันของผู้ใช้ภายในแอปพลิเคชันควรได้รับการบ่งชี้หรือระบุด้วยตัวเลขระบุเซสชันที่เฉพาะตัวไม่ซ้ำกันกับใครและ ไม่ควรมีการนำเอาเลขดังกล่าวกลับมาใช้ใหม่จากเซสชันหนึ่งไปยังอีกเซสชันหนึ่ง ถึงแม้ว่าผู้ใช้คนเดิมจะล็อกออนเข้ามาอีกครั้ง แต่ตัวเลขระบุเซสชันควรถูกสร้างขึ้นมาใหม่
2. ตัวเลขระบุเซสชันไม่ควร "ถูกคาดเดาได้"
วิธีง่ายที่สุดในการค้นหาตัวเลขระบุเซสชันที่อ่อนแอก็คือให้ทดสอบการสร้างเซสชันของผู้ใช้ขึ้นมาหลายๆคนอย่างรวดเร็วหรือเฝ้าติดตามการสร้างและทำลายเซสชันของผู้ใช้ในช่วงเวลาแคบๆ ตัวเลขระบุเซสชันที่ถูกเพิ่มขึ้นอย่างมีลำดับที่แน่นอน หรือมีค่าเปลี่ยนไปโดยอิงจากเวลาขณะนั้นหรือมีแพทเทิร์นที่คาดเดาได้ง่ายล้วนเป็นสิ่งที่ควรใส่ใจให้ส่งตัวเลขที่ถูกสร้างในลักษณะดังกล่าวมักมีความเสี่ยงต่อการโจมตีด้วยการคาดเดาค่าลำดับของตัวเลข และแฮกเกอร์สามารถเข้าครอบครองเซสชันได้จากความสำเร็จในการคาดเดาตัวเลขระบุเซสชันที่เหมือนกันกับตัวเลขระบุเซสชันของผู้ใช้ที่กำลังล็อกออนเข้ามาใช้งาน

รูปที่ 5.31 การให้ข้อมูลการป้องกัน Session Hijacking

5.7 Application Buffer Overflows

การทดลอง

เป็นการจำลองการผ่านระบบโดยกรอกชื่อผู้ใช้และรหัสผ่านซึ่งสามารถทำ Application Buffer Overflows ได้

(5.4)

```
<form action="http://www.example.com/buffercheck.php" method="post" name="form" >
<div align="center">UserName<br>
<input name="user" type="text" size="1000000" maxlength="1000000">
<span class="style1">Password</span><br>
<input name="Pwd" type="password" size="1000000" maxlength="1000000">
<input type="submit" name="Submit" value="Submit">
</div>
</form>
```


ภารกิจ
ใช้วิธีของ Application Buffer Overflows เพื่อโจมตีเว็บแอปพลิเคชัน

UserName

Password

Submit

การป้องกัน

ข้อมูลเบื้องต้น

รูปที่ 5.32 การทำ Application Buffer Overflows

เกิดจากการที่แก้ไข Filed size และ maxlength และแก้ไข filed action เพื่อส่งไปประมวลผลที่ Server

การป้องกัน

มีการตรวจสอบขนาดของอินพุตที่รับเข้ามาก่อนนำไปประมวลผล

Username หรือ Password ไม่เกิน 20 ตัวอักษร
ทดลองใหม่

[การป้องกัน](#) [ข้อมูลเบื้องต้น](#)

รูปที่ 5.33 ผลของการป้องกันการทำให้ Application Buffer Overflows

5.8 Parameter Tempering

การทดลอง

จะเป็นการทดลองโดยการRedirect ไปยังวิธีที่มีอยู่แล้วในวิธีอื่นเช่นการเปลี่ยนแปลงคำพารา

มีเตอร์ของ Cookies และ Form Fields สามารถทำการทดลองได้ที่วิธีของ Java Injection และ Hidden Manipulation ส่วน URL Query Strings สามารถทำการทดลองได้ที่วิธีของ Query Poisoning และวิธี การเปลี่ยนแปลงค่าพารามิเตอร์ของ HTTP Headers ต้องใช้เครื่องมือช่วยเหลือ เช่น burpproxy ซึ่งสามารถทำการทดลองได้กับวิธีของ Hidden Manipulation ซึ่งมีการป้องกันโดยการตรวจสอบ Referer

Parameter Tampering

ปัญหานี้เกิดจากการที่เว็บแอปพลิเคชันใช้ค่าพารามิเตอร์จากไคลเอนต์ซึ่งการเปลี่ยนค่าพารามิเตอร์นั้นสามารถทำได้ง่ายตาย ซึ่งผู้เขียนเว็บแอปพลิเคชันนั้นมักจะคิดว่าค่าต่างๆจะเป็นค่าที่ถูกต้องแล้วจะมีน้อยคนนักที่จะคำนึงถึงการเปลี่ยนแปลงพารามิเตอร์จะทำให้ระบบมีปัญหาอย่างไร ปัญหาที่เกิดขึ้นจากการเปลี่ยนค่าพารามิเตอร์ไปเป็นค่าที่ไม่ถูกต้องนั้น ทำให้เกิดความไม่ปลอดภัยในข้อมูลหลายๆ อย่างเช่นการดึงข้อมูลของลูกค้าคนอื่นๆ ได้ การเปลี่ยนสิทธิของตนเองไปเป็นของคนอื่นๆ เพื่อดึงข้อมูลส่วนตัวของคนอื่นๆ ทำให้เกิด error ขึ้นบนหน้าจอของ Web Browser ในบางที่เราสามารถเห็นถึง Source Code ที่หลุดออกมาจาก error message ที่เกิดขึ้นทำให้พาไปถึงการ Hack ระบบได้ในที่สุด

ซึ่งการทำ Parameter Tempering นั้นที่นิยมทำกันมีอยู่ 4 รูปแบบด้วยกันคือ

- Cookies ตัวอย่างการเปลี่ยนค่าพารามิเตอร์ของ Cookies ได้จากวิธี Java Injection หรือ Hidden Manipulation
- Form Fields ตัวอย่างการเปลี่ยนค่าพารามิเตอร์ของ Form Fields ได้จากวิธี Java Injection หรือ Hidden Manipulation
- URL Query Strings ตัวอย่างการเปลี่ยนค่าพารามิเตอร์ของ URL Query Strings ได้จากวิธี Query Poisoning
- HTTP Headers ต้องใช้โปรแกรมช่วยในการเปลี่ยนแปลงค่าพารามิเตอร์ของ HTTP Headers เช่น burpproxy

:: การป้องกัน ::

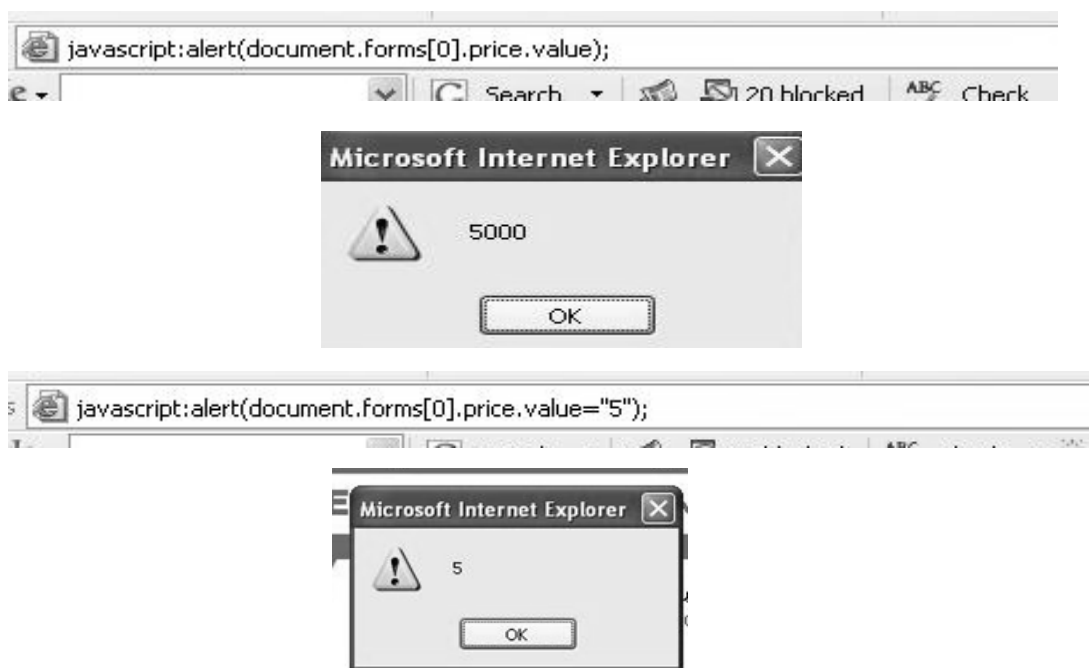
รูปที่ 5.34 หน้าการให้ความรู้และ Link ทำการทดลองของ Parameter Tempering

รูปแบบที่พบบ่อยที่ใช้เทคนิคนี้คือ

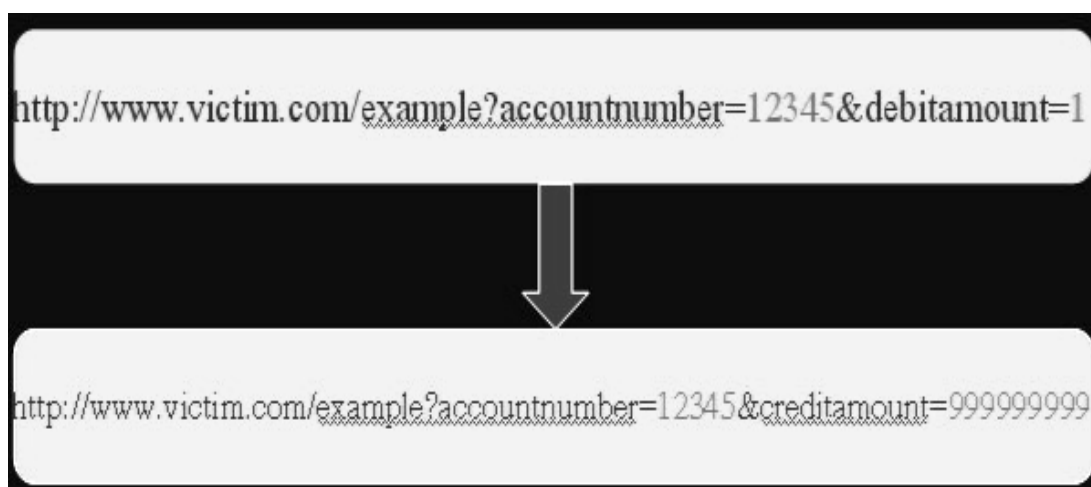
- Cookies ดังรูปที่ 5.35
- Form Fields ดังรูปที่ 5.36
- URL Query Strings ดังรูปที่ 5.37
- HTTP Headers ดังรูปที่ 5.38



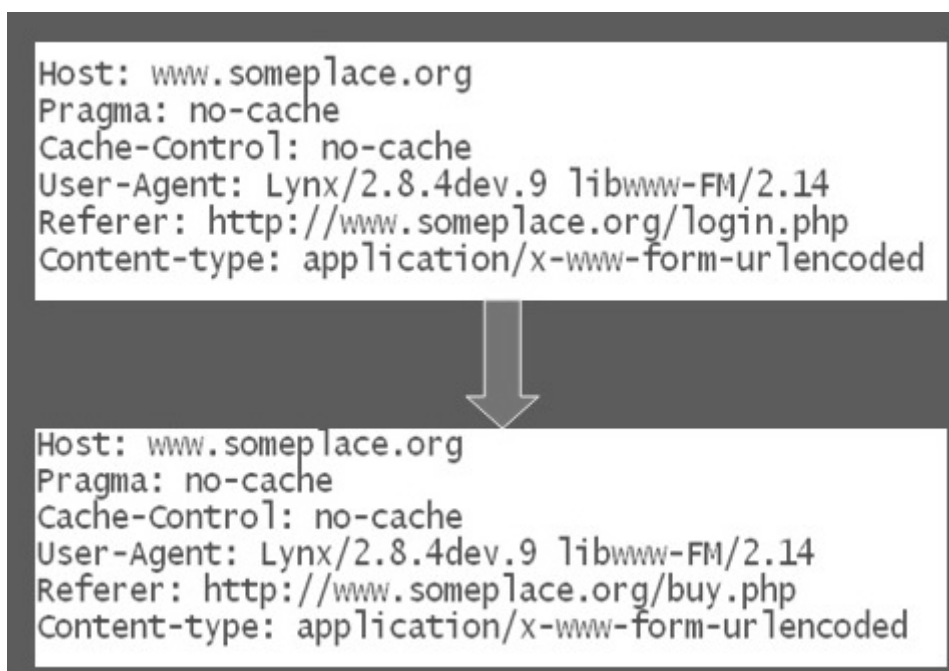
รูปที่ 5.35 เทคนิคการทำ Parameter tempering แบบ cookie



รูปที่ 5.36 เทคนิคการทำ Parameter tempering แบบ Form Field



รูปที่ 5.37 เทคนิคการทำ Parameter tempering แบบ URL Query Strings



รูปที่ 5.38 เทคนิคการทำ Parameter tempering แบบ HTTP Headers

การป้องกัน

เป็นการให้ข้อมูลในการป้องกัน

วิธีการป้องกัน Parameter Tampering

ไม่รับค่าพารามิเตอร์ที่สำคัญมาจากฝั่ง Client เนื่องจากทางฝั่ง Client นั้นสามารถที่จะเปลี่ยนค่าพารามิเตอร์ได้อย่างง่ายดาย หรือถ้ารับมาก็ต้องมีการตรวจสอบขนาดของอินพุต ให้ดีก่อนจะนำไปประมวลผล

:: กลับ ::

รูปที่ 5.39 หน้าการป้องกันของ Parameter Tempering